

# Data Structures Lab – Assignment 6

February 25, 2026

## Assignment: Sorting

### Objective:

The objective of this assignment is to implement and compare fundamental sorting algorithms including Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort. Analyze their time and space complexity under different input conditions and observe their practical behavior.

### Common Test Cases (to be used for all sorting assignments):

| Test Case                        | Input                      | Expected Output            |
|----------------------------------|----------------------------|----------------------------|
| 1. Random Order                  | {34, 12, 5, 66, 1, 89, 23} | {1, 5, 12, 23, 34, 66, 89} |
| 2. Already Sorted                | {2, 5, 8, 12, 16, 23, 38}  | {2, 5, 8, 12, 16, 23, 38}  |
| 3. Reverse Sorted                | {50, 40, 30, 20, 10, 5, 1} | {1, 5, 10, 20, 30, 40, 50} |
| 4. With Duplicates               | {4, 2, 7, 2, 9, 4, 1}      | {1, 2, 2, 4, 4, 7, 9}      |
| 5. Negative and Positive Numbers | {-5, 12, 0, -3, 8, -1}     | {-5, -3, -1, 0, 8, 12}     |

### 1. Student Marks Ranking (Bubble Sort)

**Scenario:** A school teacher wants to rank students based on their exam marks in ascending order so that the lowest scorer can be identified for remedial classes.

#### Task:

- Input: List of student marks.
- Use Bubble Sort to sort marks in ascending order.
- Display:
  - Sorted marks
  - Total number of swaps performed
  - The topper and lowest scorer

#### Extension:

- Modify Bubble Sort with an early stopping condition.

- Compare number of passes when the list is already sorted.

## 2. Product Price Comparison Tool (Selection Sort)

**Scenario:** An e-commerce startup wants to display products sorted by price so customers can compare cheapest options first.

**Task:**

- Input: List of product prices.
- Use Selection Sort to sort prices in ascending order.
- Print the list after each outer loop iteration.

**Extension:**

- Count number of comparisons made.
- Explain why Selection Sort makes fewer swaps than Bubble Sort.

## 3. Event Registration by Timestamp (Insertion Sort)

**Scenario:** An event management system records participant registration times. Registrations must be displayed in chronological order.

**Task:**

- Input: List of timestamps.
- Use Insertion Sort to sort them.
- Demonstrate why Insertion Sort is efficient for **nearly sorted data**.

Note: A nearly sorted data (also known as a  $k$ -sorted array or almost sorted array) is an array in which every element is at most  $k$  positions away from its correct sorted position. This means that if the array were completely sorted, an element found at index  $i$  in the original (nearly sorted) array could be at any index from  $i - k$  to  $i + k$  in the final sorted array.

**Extension:**

- Count number of shifts performed.

## 4. Merging Two Sorted Branch Sales Reports (Merge Procedure)

**Scenario:** Two branches of a company submit their sales data already sorted. The head office needs to merge them into a single sorted report.

**Task:**

- Input: Two sorted arrays.
- Implement the merge procedure of Merge Sort.
- Output merged sorted array.

**Extension:**

- Count number of comparisons during merging.

## 5. National Exam Ranking System (Merge Sort)

**Scenario:** A national-level exam has thousands of candidates. Efficient sorting is required to generate rank lists.

**Task:**

- Input: Large list of candidate scores.
- Implement Merge Sort recursively.
- Display:
  - Time taken for sorting
  - Discussion of space complexity

**Extension:**

- Compare performance with Bubble Sort for large inputs.

## 6. E-Commerce Flash Sale Sorting (Quick Sort)

**Scenario:** During a flash sale, product ratings must be quickly sorted to show top-rated products first.

**Task:**

- Input: List of product ratings.
- Implement Quick Sort using first element as pivot.
- Print array after each partition step.

**Extension:**

- Try:
  - Last element as pivot
  - Random pivot
- Compare performance.